Concurrency and Performance

compare-and-swap (**CAS**), or in C++ as `compare_exchange`. It takes two parameters: the first is the expected current value of the atomic variable, and the second is the desired new value.

If the actual current value does not match the expected one, nothing happens; there is no change to the atomic variable. However, if the current value does match the expected one, the desired value is written into the atomic variable.

The C++ `compare_exchange` operation returns *true* or *false* to indicate whether the write happened (*true* if it did). If the variable did not match the expected value, the actual value is returned in the first parameter. With `compare-and-swap`, atomic increment operation can be implemented in the following way:

```
std::atomic<size_t> count;
… on the threads …
size_t c = count.load(std::memory_order_relaxed);
while (!count.compare_exchange_strong(c, c + 1,
  std::memory_order_relaxed, std::memory_order_relaxed)) {}
```

First, the actual name of the operation in C++ is `compare_exchange_strong`. There is also `compare_exchange_weak`; the difference is that the weak version can sometimes return `false` even when the current and expected values match (on x86, it makes no difference, but on some platforms, the weak version can result in a faster overall operation).

Second, the operation takes not one but **two** memory order arguments: the second applies when the comparison **fails** (so it is the memory order for just the **load** part of the operation). The first applies when the comparison succeeds and the write happens.

How this implementation of the increment works? First, the current value of the **count**, `c` is atomically read. The incremented value is, of course, `c + 1`, but `count` cannot be assigned by this value because another thread could have incremented the `count` after the read but before the update. So a conditional write is needed: if the current value of the count is still `c`, replace it with the desired value `c + 1`. Otherwise, update `c` with the new current value (`compare_exchange_strong` does that) and try again. The loop exits only when the atomic variable did not change between the last time to read it and the time of being updated. This approach can be generalized to any computation: instead of `c + 1`.

Which version of `compare_exchange` should be used, and which memory order applies? On most CPUs, `compare_exchange` is a single atomic hardware instruction, and conceptually, the execution flow of `compare_exchange` looks like

```
template <typename T> class atomic {
  mutex m;
  T value;
  bool compare_exchange_strong(T& old_v, T new_v) {
    std::lock_guard l(m);      // Get exclusive access
    T tmp = value;             // Current value of the atomic
    if (tmp != old_v) { old_v = tmp; return false; }
    value = new_v;
    return true;
  }
};
```

But this is not the most efficient way to implement `compare_exchange`. Generally, exclusive access to simply read the shared value is not needed. Of course, if another thread is writing the same value, the result of unlocked read can be wrong and needs to be verified under a lock, but only if the first read returns the expected current value and the compare-and-swap can proceed:

```
template <typename T> class atomic {
  mutex m;
  T value;
  bool compare_exchange_weak(T& old_v, T new_v) {
    T tmp = value;           // Current value of the atomic
    if (tmp != old_v) { old_v = tmp; return false; }
    std::lock_guard l(m);    // Get exclusive access
    tmp = value;             // True current value of the atomic
    if (tmp != old_v) { old_v = tmp; return false; }
    value = new_v;
    return true;
  }
};
```

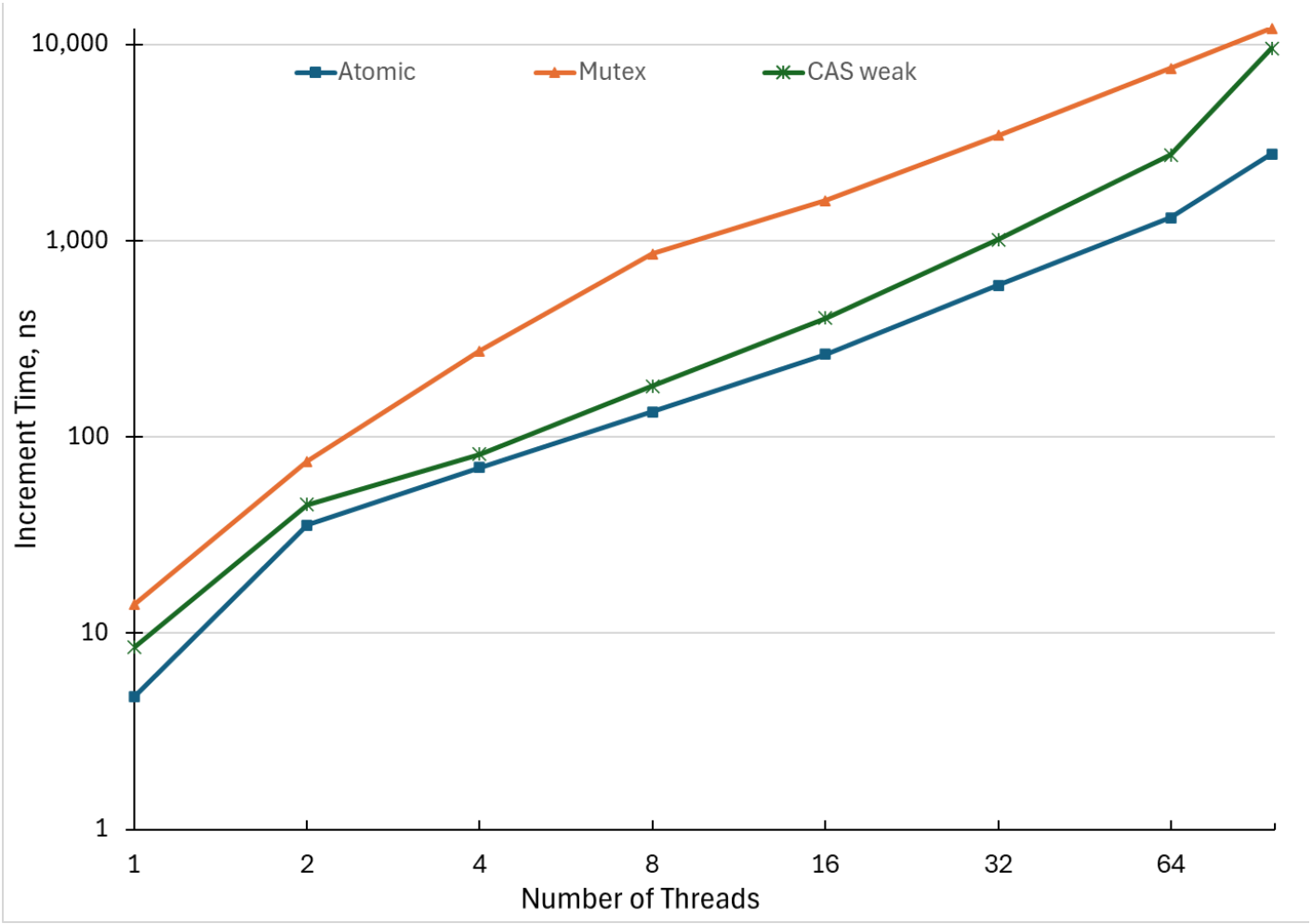
This is the classic `double-checked` locking pattern. It quickly return `false` in most cases when the current value does not match the expectation. However, there is a risk: it may return `false` based on a stale unlocked read, whereas the accurate comparison, done under a lock, would have returned `true` (and carried out the exchange).

Furthermore, some processor architectures, notably ARM, have the equivalent of a timed mutex, where the exclusive lock can fail if it takes too long to acquire. In this case, `compare_exchange_weak` also returns `false`, even if the atomic value is expected. Why yield the lock when it should proceed with the exchange? On CPUs with many cores, making everyone wait as long as it takes reduces the overall throughput; it is better to yield to another thread so it can get the lock faster and proceed with its computation.

Finally, why does `compare_exchange` take two memory order arguments? Take another look at the pseudocode: there are two separate operations on the atomic variable. Its value is always read at least once. That read operation may have a memory barrier. If the comparison leads to an exchange, the operation may have a different memory barrier that is only engaged if `compare_exchange` succeeds. Hence, two memory order arguments: one for success and one for read-without-write.

Now add the compare-and-swap loop to the benchmark and compare it with the other two ways to increment a shared variable:

```
// Example 01
// CAS increment
std::atomic<unsigned long> xa(0);
void BM_cas_weak(benchmark::State& state) {
  std::atomic<unsigned long>& x = xa;
  for (auto _ : state) {
    unsigned long xl = x.load(std::memory_order_relaxed);
    while (!x.compare_exchange_weak(xl, xl + 1,
            std::memory_order_relaxed,
            std::memory_order_relaxed)) {}
  }
  state.SetItemsProcessed(state.iterations());
}
```



Lock-based, Lock-free, and Wait-free programs

In lock-based (with mutex) program, only one thread can hold the lock at any time, so that thread can increment the count without any further precautions. Once the lock is released, another thread can acquire it and increment the count. At any time, at most one thread can hold the lock and make progress; all remaining threads that need access are waiting on the lock. But even the thread that has the lock is not guaranteed to proceed: if it needs access to another shared variable before it can complete its job, it may be waiting on that lock, which is held by some other thread.

In atomic based increment program, any thread that arrives at the atomic increment operation executes it without delay. Naturally, the hardware itself must serialize access to the shared data to ensure the atomicity of the operations (this is done by granting exclusive access to the entire cache line to one processor at a time). From the programmer's perspective, this exclusive access manifests as an increase in the time it takes to execute the atomic operation. However, in the code itself, there is no waiting for anything, no trying and retrying. This kind of program is called **wait-free**. In a wait-free program, all threads are making progress—that is, executing operations—at all times (although some operations may take longer if there is severe contention between threads for access to the same shared variable). A wait-free implementation is usually possible only for very

simple operations (like incrementing a count), but whenever it is available, it is even simpler than the lock-based implementation.

In the `cas` based implementation, there are no locks; however, there is a loop that repeats an unknown number of times. In this regard, the implementation is similar to a lock: any thread waiting on a lock is also stuck in a similar loop, trying and failing to acquire the lock. However, there is one key difference: in a lock-based program, when a thread can acquire lock is not guranteed. In the program based on compare-and-swap, the only way a thread can fail to update the shared count is because some other thread updated it first. Therefore, of all threads trying to increment the count at the same time, at least one will always succeed. This kind of program is known as **lock-free**.

The above analysis shows three main types of concurrent programs:

- In a **wait-free** program, each thread is executing the operations it needs and is always making progress toward the final goal; there is no waiting for access.
- In a **lock-free** program, multiple threads may be trying to update the same shared value, but only one of them will succeed. The rest will have to discard the work they have already done based on the original value, read the updated value, and do the computation again. But at least one thread is always guaranteed to commit its work and not have to redo it; thus, the entire program is always making progress, although not necessarily at full speed.
- Finally, in a **lock-based** program, one thread is holding the lock that gives it access to the shared data. Just because it's holding the lock does not mean it is doing anything with this data, though. So, when concurrent access happens, at most one thread is making progress, but even that is not guaranteed.

Different locks for different problems

Usually, standard `C++ mutex` on most platforms performs very poorly when it is used to guard access to a shared variable, especially when there are many threads trying to modify this variable at the same time. But is the lock inefficient because of its implementation, or is the problem inherent in the nature of the lock? Any lock is expected to be somewhat less efficient than the atomically incremented counter simply because a lock-based scheme uses two shared variables (the lock and the count) versus just one shared variable for an atomic counter. However, the mutexes provided by the operating system are usually not particularly efficient for locking very short operations .

The simplest and one of the most efficient locks for this situation is a basic **spinlock**. The idea of the spinlock is this: the lock itself is just a flag that can have two values, normally are `0` and `1`. If the value of the flag is `0`, the lock is **not locked**. Any thread that sees this value can set the flag to `1` and proceed. The entire operation to read the flag and set it to `1` has to be a **single atomic operation**. Any thread that sees the value of `1` must wait until the value changes back to `0` to indicate that the lock is available. Finally, when a thread that changed the flag from `0` to `1` is ready to release the lock, it changes the value back to `0`.

The simplest implementation of this lock looks like this:

```
// Example 02
class BasicSpinlock {
  public:
  void lock() {
    while (flag_.exchange(1, std::memory_order_acquire)) {}
  }
  void unlock() {
    flag_.store(0, std::memory_order_release);
  }
  private:
  std::atomic<unsigned int> flag_ {};
};
```

The above implementation shows only the locking and unlocking functions; the class also needs declarations that make it **non-copyable**.

Note that locking the flag does not use a conditional exchange, but always write 1 into the flag. The reason it works is that if the original value of the flag was `0`, the exchange operation sets it to `1` and returns `0` (and the loop ends). But if the original value was 1, it is replaced by 1 so it does not change at all.

Also, note the two memory barriers: locking is accompanied by the **acquire** barrier, while unlocking is done with the **release** barrier. Together, these barriers delimit the critical section and ensure that any code written between the calls to `lock()` and `unlock()` stays there.

However, the performance of this spinlock is terrible. To make it useful, it needs several optimizations.

First, note that if the value of the flag is `1`, the flag is left alone as it is. Why does it matter? The exchange is a `read-modify-write` operation; even if it changes the old value to the same value, it needs exclusive access to the cache line containing the flag. but exclusive access is not necessary to just read the flag.

This matters in the following scenario: a lock is locked, the thread that has the lock is not changing it (it is busy doing its work), but all other threads are checking the lock and waiting for the value to change to 0. If they do not try to write into the flag, the cache line does not need to bounce between different CPUs: they all have the same copy of the memory in their caches, and this copy is current, so there is no need to send any data anywhere. Only when one of the threads actually changes the value does the hardware need to send the new content of the memory to all CPUs.

Here is the optimization

```
// Example 02
class LoadSpinlock {
  void lock() {
    while (flag_.load(std::memory_order_relaxed) ||
           flag_.exchange(1, std::memory_order_acquire)) {}
  }
}
```

The optimization here is that the flag is read until the value `0` is observed; then, the flag's values is swapped by `1`. The value could have changed to 1 between the time of check and the time when the exchange is called if another thread got the lock first. Also, note that when pre-checking the flag, the memory barrier doesn't matter since the final definitive check is always done using the exchange and its memory barrier.

Even with this optimization, the lock performs poorly. The reason has to do with the way operating systems tend to prioritize threads. In general, a thread that is doing heavy computing will get more CPU time on the assumption that it is doing something useful. Unfortunately, in this case, the most heavily computing thread is the one hammering on the flag while waiting for it to change. This can lead to an undesirable situation where one thread is trying to get the lock and has the CPU allocated to it, while another thread would like to release the lock but doesn't get scheduled for execution for some time. The solution is for the waiting thread to relinquish the CPU after several attempts so some other thread can run to finish its work and release the lock.

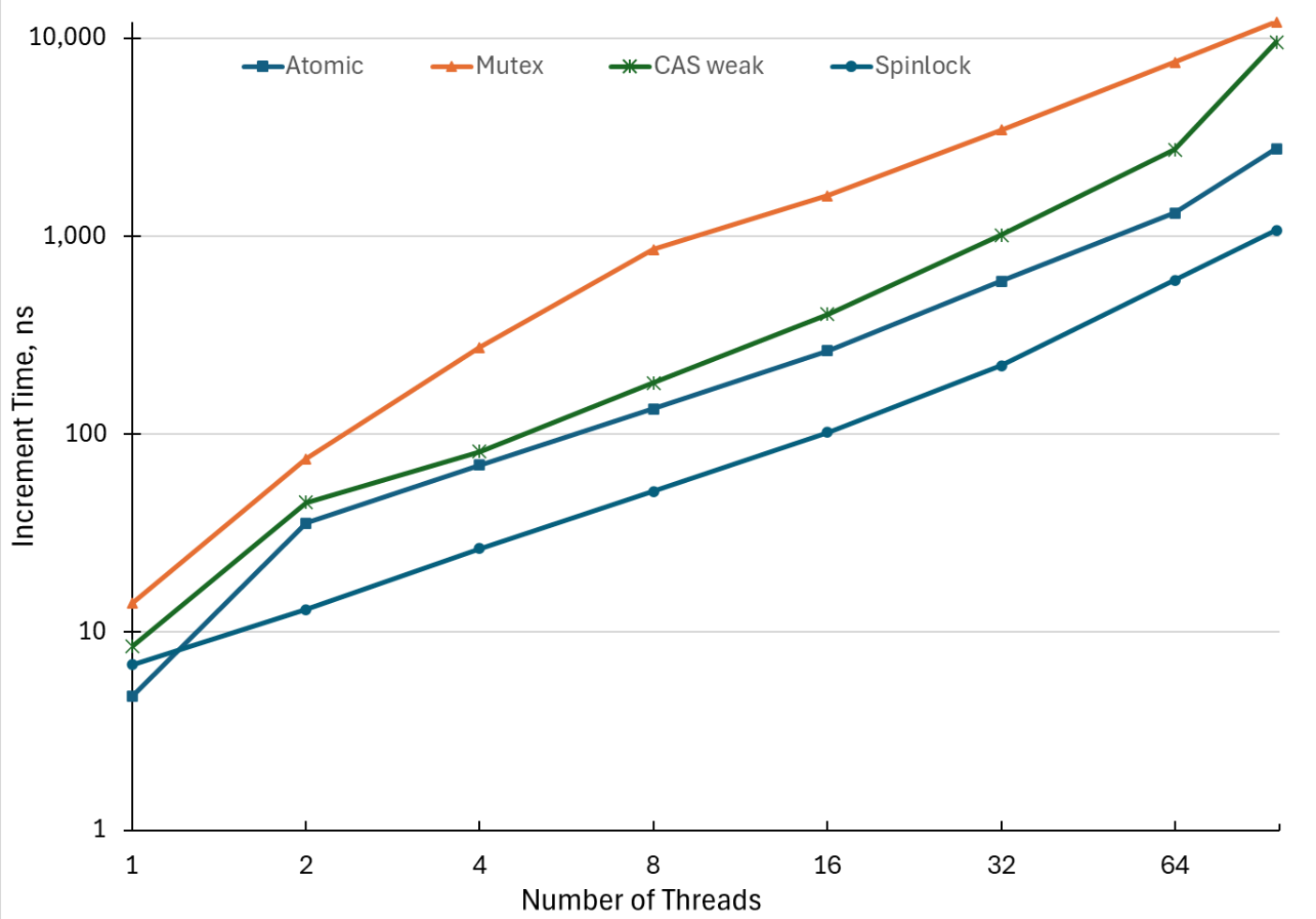
There are several ways for a thread to relinquish control of the CPU; most involve a system call. There isn't a universal best way to do so. The most common ways are the CPU pause instruction (emitted by inline assembly or compiler intrinsic), a call to `std::this_thread::yield()` (on Linux, this calls `sched_yield()`), or a call to sleep for a very short time (1 nanosecond) by calling `nanosleep()`. Note that while pause is a cheap hardware instruction, yield and sleep are system calls, which are expensive compared to hardware instructions. The best balance is achieved when trying to get the lock several times, then yield the CPU to another thread, then try again:

```
// Example 02
class Spinlock {
  void lock() {
    for (int i=0; flag_.load(std::memory_order_relaxed) ||
      flag_.exchange(1, std::memory_order_acquire); ++i) {
      if (i == 8) {
        static const timespec ns = { 0, 1 }; // 1 ns
        nanosleep(&ns, nullptr);
        i = 0;
      }
    }
```

```
    }
  }
```

The optimal number of attempts to acquire the lock before releasing the CPU will depend on the hardware and the number of threads, but generally, values between 8 and 16 work well. The best practice is to benchmark different implementations (pause, yield, sleep, or even a few pauses followed by a sleep) to see which one works best.

Here are the benchmark results:



The spinlock has done very well: it is soundly outperforming the compare-and-swap implementation and gives the wait-free operation tough competition (the atomic increment is faster on one thread, where there is no contention, but slower otherwise).

Why does the spinlock perform so much better, specifically the lock with the `nanosleep` backoff? The reason is found in the low-level details of the cache coherency protocol.

When a thread acquires the lock, it does so by successfully writing 1 into the lock flag. To do this, the core must first obtain exclusive ownership of the cache line containing the flag. If other cores

are holding it, it issues an invalidation request (often a Read-For-Ownership) to transition the line into the **Exclusive** (E) state. Once it holds the line exclusively, it performs the write, which transitions the line into the **Modified** (M) state.

When other cores try to acquire the lock, they first read the flag. This read request forces the owning core to write the dirty line back (or supply it directly) and downgrade its state from **Modified** to Shared (**S**).

This downgrade itself is relatively cheap. This is exactly why a Test-and-Test-and-Set (TTAS) lock reads first: if we instead used a naive atomic exchange on every iteration, every core would constantly demand the cache line be moved back to the **Exclusive** state just to get permission to write, even if all it does is replace a 1 with a 1.

However, the hidden cost of contention is accrued when the owning thread needs to unlock. Unlocking is a simple, unconditional write of 0. But to execute that write, the cache line must be upgraded from **Shared** back to **Exclusive**. The writing core must send an invalidation request to all other cores holding the line in **S**. In smaller CPUs, this is done via a simple broadcast, but this approach does not scale past 16 cores or so: full broadcast consumes massive bandwidth. In a server, dissipating the resulting heat is a major concern, and sustained full broadcasts to dozens of cores would literally melt the interconnect fabric.

Large x86 and ARM server chips use more complex network topologies (like ring buses or meshes) and snoop filters that track which core's caches hold specific lines to avoid chip-wide broadcasts and thus reduce bandwidth and power consumption. But nothing can circumvent the latency of waiting for acknowledgment (ACK) signals from all the cores that previously read the lock and currently hold the cache line in the **Shared** state.

The pause instruction does not help here. A paused thread puts its core into a short sleep (around 50 cycles), but the cache remains completely valid. The hardware still has to collect ACK signals from all paused cores, because a paused core still responds immediately to coherency probes—only its instruction execution is paused.

In contrast, `nanosleep` removes the thread from the OS scheduler entirely. The crucial factor is not what happens to the cache during the context switch, but the fact that the sleeping thread is no longer actively executing the spin loop. When the lock owner finally writes 0 and forces the cache line back to the Exclusive state, the sleeping threads are not awake to immediately re-read the flag and drag the line back to the Shared state. This aggressive backoff breaks the cycle. It drastically reduces the number of cores actively fighting for the cache line. At high contention, the invalidation broadcast and response traffic dominate execution time; reducing the number of cores actively participating in this sharing is the single optimization that matters above everything else.

These results and their understanding leave two questions: First, why don't all locks use spinlocks if they are so much faster? Second, why atomic operations are needed if the spinlock is so good?

The answer to the first question is different locks for different problems. The downside of the spinlock is that the waiting thread continuously uses the CPU, or is `busy waiting`. In contrast, a thread waiting on a system mutex is mostly idle (sleeping). Busy waiting is great if you need to wait for just a few cycles, such as the duration of an increment operation: it's much faster than putting the thread to sleep. However, if the locked computation consists of more than a handful of instructions, the threads waiting on the spinlock waste a lot of CPU time and deprive the other working threads of access to the hardware resources they need.

The second question introduce the difference between lock-based and lock-free.

Lock-based versus Lock-Free, what is the real difference?

When the conversation turns to the advantages of lock-free programming, the first argument is usually `it is faster`. However, this is not necessarily true: lock implementations can be very efficient if optimized for a particular task.

The issue of overhead on both lock-based and lock-free implementation are clear: spinning loops found in spinlocks and CAS-based lock-free programs can consume large amounts of CPU time while waiting. **If the expected wait is long and the contention is high, a lock may be more efficient**. However, if threads spend a lot of time waiting to complete a computation that can be done only on one thread at a time, the program is effectively serial at that point. A mutex will likely have less overhead than a CAS loop, but since there is no parallelism, the fastest option with zero overhead is simply to do the work on one thread. Naturally, this entire situation is to be avoided, but the solution usually involves finding an algorithm with more inherent parallelism.

Even **near-instant** access to shared variables puts significant stress on the CPU hardware, and this comes at a cost. So far, we have measured the throughput of the operation on a shared variable itself, but this is unlikely to be the main purpose of your program. (As we just pointed out, the best-case scenario for the performance of exclusive access is single-threaded speed, so if such access is all you do, you cannot benefit from threading.)

Most real programs access shared data in between independent parallel computations. We can construct a benchmark that emulates such a workload by interleaving shared data access with work that is local to each thread. For example, here is a benchmark that combines atomic shared increments with some independent computations:

```
// Example 03
std::atomic<unsigned long> xa(0);
void BM_atomic(benchmark::State& state) {
  const size_t nshared = state.range(0);
  const size_t nwork = state.range(1);
  std::atomic<unsigned long>& x = xa;
  double y = 1;
```

```
  unsigned long n = 0;
  for (auto _ : state) {
    for (size_t i = 0; i != nshared; ++i) {
      benchmark::DoNotOptimize(x.fetch_add(1,
        std::memory_order_relaxed));
    }
    for (size_t i = 0; i != nwork; ++i) {
      benchmark::DoNotOptimize(y = sin(++n + cos(y)));
    }
  }
  state.SetItemsProcessed(n);
}
```

Note that when we count `items` of work, we only count the work done in parallel (the `sin` / `cos` calculations tracked by `n`). This mimics the common situation where data sharing is an implementation detail done `in service` of the computations that lead to the results we care about.

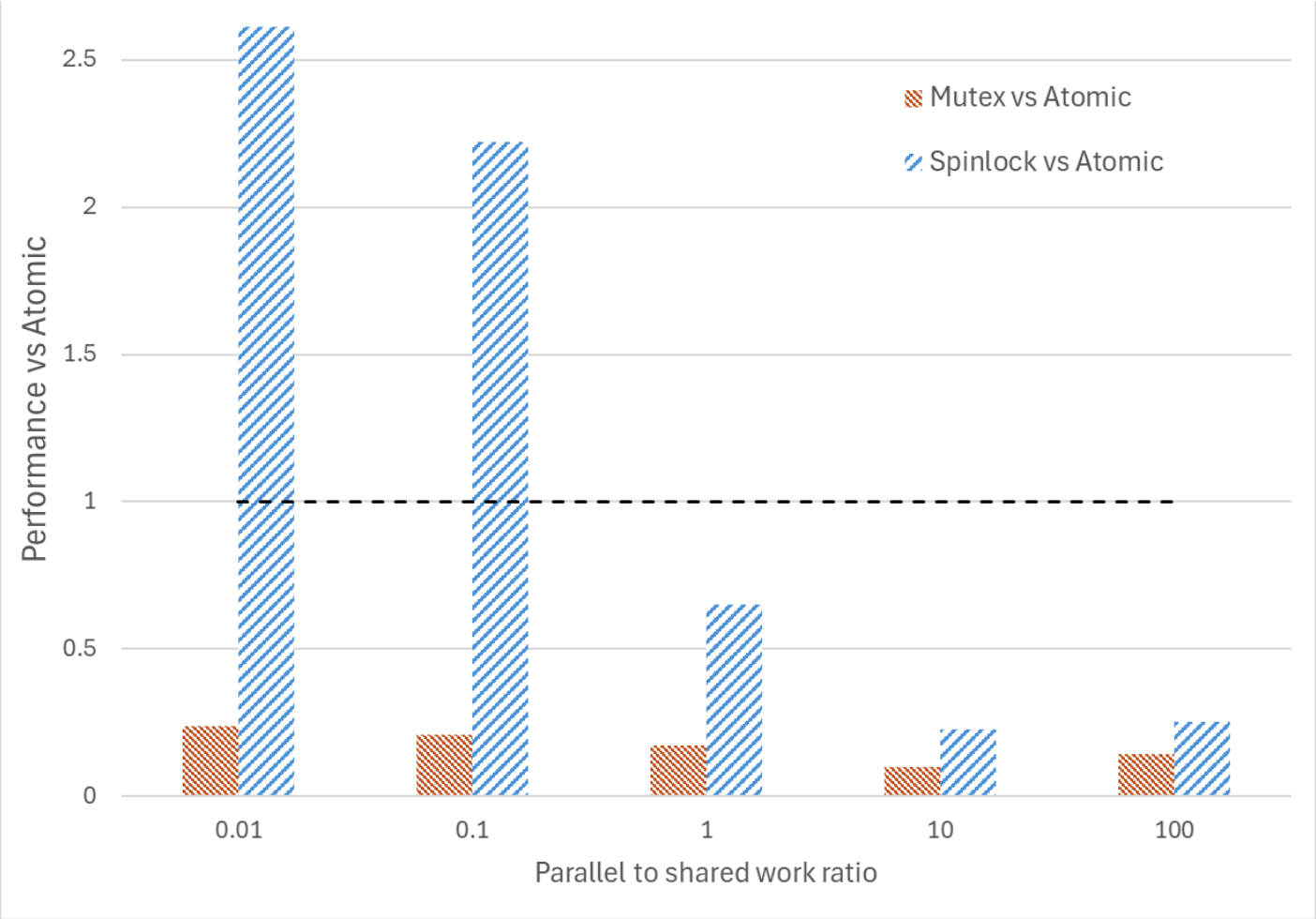
We can now compare the performance of different ways to provide exclusive access to the shared data. If the shared variable is guarded by a spinlock, we need to change the first loop to:

```
// Example 03
unsigned long x = 0;
Spinlock S;
// … inside the benchmark loop …
    for (size_t i = 0; i != nshared; ++i) {
      std::lock_guard L(S);
      benchmark::DoNotOptimize(++x);
    }
```

We can similarly use a **mutex** instead of the *spinlock*. We then invoke this benchmark with different ratios of shared vs. independent workloads:

```
// Example 03
static const long numcpu = sysconf(_SC_NPROCESSORS_CONF);
#define ARGS ->Threads(numcpu)->Args({1, 1}) \
  ->Args({1, 10})->Args({1, 100}) \
  ->Args({10, 1})->Args({100, 1}) \
  ->UseRealTime()
BENCHMARK(BM_atomic) ARGS;
BENCHMARK(BM_mutex) ARGS;
BENCHMARK(BM_spinlock) ARGS;
```

The outcomes depend on the processor and the number of threads. The following figure shows the results for the **Intel Sapphire Rapids** processor with 48 cores (96 threads) running one thread per *logical core*.



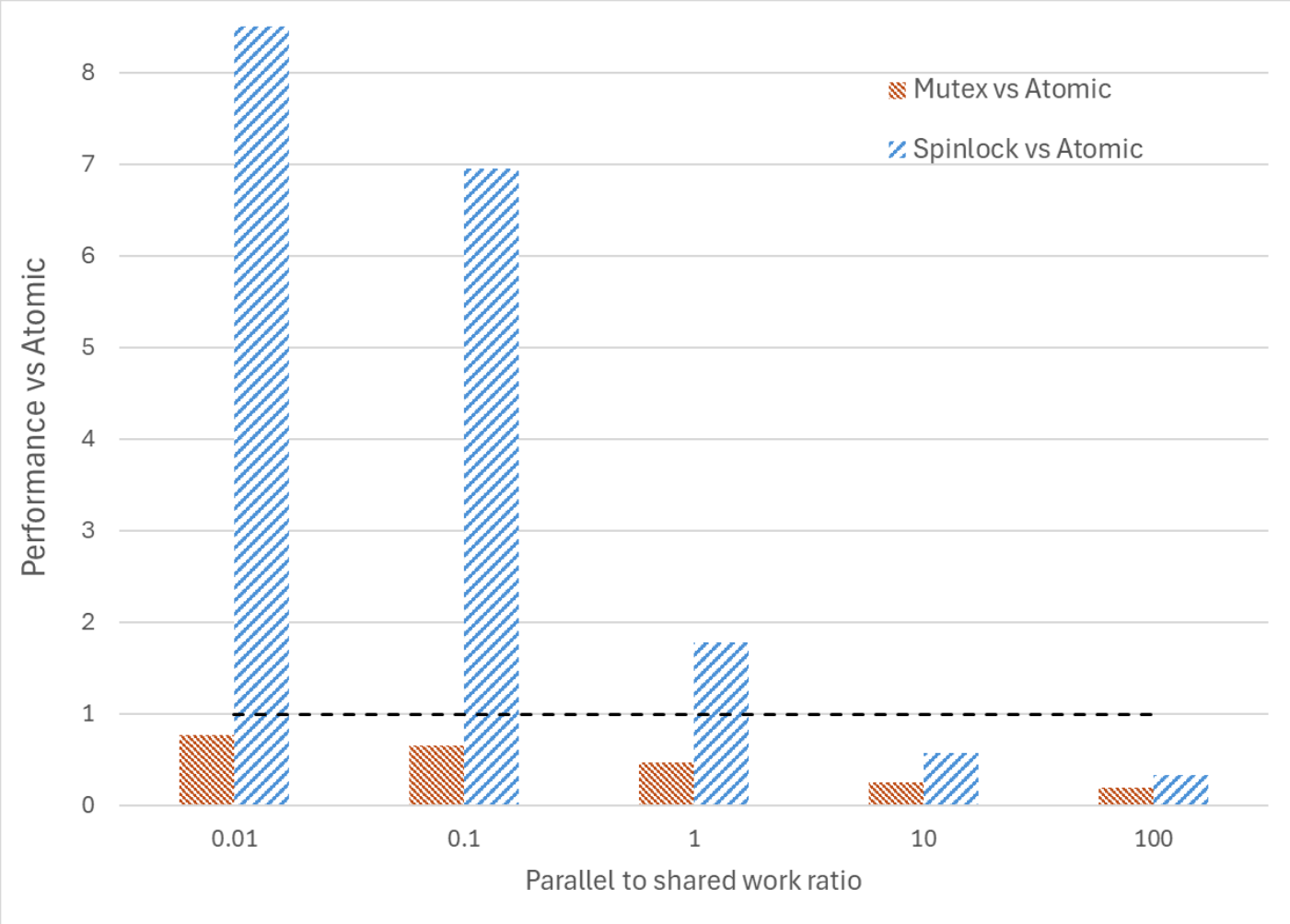
The most interesting result here is that, while the spinlock consistently provides better throughput for the shared access itself (the total number of increments of the shared variable per second), it is not always the best choice for the overall throughput of a program that uses shared data as needed to run large amounts of per-thread, independent computations. When **contention for the shared data is low**, atomic operations have lower overhead; the spinlock uses more CPU time, leaving fewer computing resources for the useful work (which in our benchmark is modeled by the independent workload computed by each thread). Only when the **contention is high** and **per-thread work is often interrupted to access the shared variable**, the performance of the spinlock is better. The mutex remains much slower and less efficient across the board.

While this trend is universal, the **threshold** contention depends both on the hardware and on the specific instructions used to access the shared data. It is also worth asking whether the same trend (atomic operations have less overhead at low contention) holds if we lack a dedicated atomic instruction and must use a **CAS** loop. We don't need a complex example for this: something as simple as multiplication or computing the maximum value cannot be done atomically and requires

a CAS loop. In this benchmark, to keep our results comparable, we will use CAS to implement the same increment:

```
// Example 03
// ... inside the benchmark loop ...
    for (size_t i = 0; i != nshared; ++i) {
      unsigned long xl = x.load(std::memory_order_relaxed);
      while (!x.compare_exchange_strong(xl, xl + 1,
        std::memory_order_relaxed,
        std::memory_order_relaxed)) {}
      benchmark::DoNotOptimize(xl);
    }
```

As expected, the CAS loop perform worse than the atomic instruction. The question is: Will it outperform the spinlock at low contention?



From the above figure, CAS outperform the spinlock at low contention.

This is a rather striking turnaround from the conventional wisdom derived from measurements obtained on hardware just a few years ago: **lock-free code can be more efficient, but its domain is that of low contention**. On the Sapphire Rapids CPU with 96 threads, the overhead of the lock (of any kind, spinlock or mutex) is significant even when the locked section is just *1%* of the total computation. On the 256-thread EPYC CPU, spinlock-based code is **slower** than lock-free code even when the locked section is *0.1%* of the total (only around *0.01%* does the locking become so `diluted` that its overhead is insignificant).

Amdahl's law can at least partially explain this: the cost of the critical section is essentially multiplied by the number of threads. But we already know that atomic operations are also executed one at a time; the only difference is that the hardware handles the exclusive access. The second reason is that the lock-free code is more predictable and, therefore, more pipeline-friendly. CPUs analyze the code dozens of instructions ahead, but they can't do it across the critical section boundary (or at least cannot do it as well).

This explanation can be confirmed by direct measurements using hardware performance counters. In particular, we want to measure the **core-bound stalls**, which tell us how often the execution unit is halted because it must wait for a core event to complete.

On Intel x86 CPUs, the performance counter `cycle_activity.stalls_total` measures the aggregate stalls. We can collect it, along with the generally useful cycle and instruction counts, using this `perf` command on Linux:

```
perf stat \
  -e cycles,instructions,cycle_activity.stalls_total \
  ./benchmark
```

For these tests, we want to run only one benchmark at a time: one contention ratio and one synchronization mechanism. We should also note that, at low contention, replacing `nanosleep` with a pause instruction (`_mm_pause()` on x86) yields slightly better throughput and much cleaner benchmark results, as it avoids profiling the system call overhead.

The results show that the execution unit is starved of work much more often when a spinlock is used, but it doesn't immediately tell us *why*. What it does explain is the observed performance difference, where the lock-based code runs at around **30%** of the speed of the lock-free code. It is not that the shared data updates take that much longer: at low contention, there are not enough of them to cause that much slowdown even if the lock takes several times longer than the atomic operation. The problem is that the lock somehow disrupts the execution pipeline for the perfectly independent, thread-local parallel code. In effect, spinlocks *poison* the execution unit, dragging down the performance of the surrounding code.

There are several suspects here:

1. **Branch Misprediction**: Both spinlock and CAS versions use loops and rely on the branch predictor. We expect the predictor to be highly accurate at low contention, but this needs confirmation.
2. **Store Buffer Stalls**: Store buffers hold the results of write operations locally in the core, allowing the CPU to execute subsequent instructions without waiting for data to move into the caches, coalescing results before flushing to memory. Atomic operations (including CAS) use the shared variable itself for synchronization. In comparison, a lock relies on an implicit execution dependency between at least two distinct variables: the lock flag and the data it guards.
3. **Instruction Reordering Limits**: We may suspect that the strict execution order required by the spinlock (acquire the lock → modify state → release the lock) restricts the CPU's ability to reorder instructions

Instructions per cycle	**Reorder Buffer Stalls**	**Store Buffer Stalls**	**Total Stalls**
Atomic add	1.23	2,665,943,420	12,074	59,294,793,668
CAS	1.15	4,066,931,054	13,177	64,094,964,960
Spinlock	0.38	1,537,175,087	1,587,237,048	130,257,880,756